

22-monorepo

22 — Monorepo: Satu Repo Banyak Project

□ Pengantar

Pernah kerja project web + API di repo terpisah? Repot. Tipe data beda, CI beda, refactor harus buka 2 repo. Monorepo jawabannya: **semua kode di satu repository**, tetap terisolasi per project.

Monorepo ≠ monolith. Kamu tetap punya banyak aplikasi kecil, tapi semua tinggal serumah. Digunakan oleh Google, Meta, Uber, dan tim produksi modern.

1. Kenapa Monorepo?

□ Shared types (tidak perlu duplikasi)

Tanpa monorepo:

```
// apps/web/types.ts
interface User { id: string; name: string; email: string; }
```

```
// apps/api/types.ts
interface User { id: string; name: string; email: string; }
```

Satu ubah field, yang lain lupa — bug runtime. Dengan monorepo:

```
// packages/shared/src/types.ts
export interface User {
  id: string;
  name: string;
  email: string;
}
```

```
// apps/web/ & apps/api/ — import dari satu tempat
import { User } from '@project/shared';
```

Satu sumber kebenaran.

□ Satu CI/CD

Configure sekali — semua project kena. Build, lint, test otomatis. Tidak perlu setup CI 3x terpisah.

□ Refactor lebih mudah

Ganti nama field email jadi emailAddress? Cari & ganti sekali di root. Git blame satu repo. Perubahan bisa atomic commit.

□ Dependency management terpusat

Versi TypeScript, ESLint, Prettier — cukup di root package.json. Semua project pakai sama. Zero konflik.

2. Struktur Project Monorepo

```
monorepo-app/
├── apps/
│   ├── web/                # Frontend (Next.js, React, dll)
│   │   ├── package.json
│   │   └── src/
│   └── api/                # Backend (Express, Hono, NestJS)
│       ├── package.json
│       └── src/
├── packages/
│   ├── shared/            # Types, utils, interfaces
│   │   ├── package.json
│   │   └── src/
│   └── eslint-config/     # Konfigurasi ESLint bersama
│       ├── package.json
│       └── index.js
├── package.json           # Root workspace
├── turbo.json             # Turborepo config
└── .gitignore
```

- apps/ — aplikasi yang bisa dijalankan (web, API, mobile)
 - packages/ — pustaka internal yang dipakai oleh apps
 - Root package.json — deklarasi workspace saja
 - turbo.json — pipeline & caching
-

3. NPM Workspaces

NPM Workspaces adalah fitur bawaan npm/ yarn/ pnpm. Membolehkan instalasi dependensi sekali untuk semua package.

Setup

Root `package.json`:

```
{
  "private": true,
  "name": "monorepo-app",
  "workspaces": [
    "apps/*",
    "packages/*"
  ],
  "scripts": {
    "dev": "turbo dev",
    "build": "turbo build",
    "lint": "turbo lint"
  }
}
```

- "workspaces" — array glob pattern. `apps/*` artinya semua folder di dalam `apps/` adalah workspace.
- "private": true — root tidak boleh dipublish ke npm.
- Script root cukup panggil Turborepo, nanti Turborepo urus eksekusi per workspace.

Hoisting

NPM akan "menaikkan" (hoist) dependensi bersama ke `node_modules` root. Dependensi yang sama (React, TypeScript) cukup sekali download. Ini hemat disk space dan bikin install lebih cepat.

Setiap workspace punya `package.json` sendiri

```
// packages/shared/package.json
{
  "name": "@myapp/shared",
  "version": "1.0.0",
  "main": "dist/index.js",
  "types": "dist/index.d.ts",
  "scripts": {
    "build": "tsc"
  }
}
```

```
// apps/web/package.json
{
  "name": "@myapp/web",
  "version": "1.0.0",
  "dependencies": {
    "@myapp/shared": "*",
    "react": "^18.0.0"
  },
  "scripts": {
    "dev": "next dev",
    "build": "next build"
  }
}
```

"*" artinya pakai versi berapa pun yang ada di lokal workspace — npm langsung symlink ke packages/shared.

4. Turborepo: Pipeline & Caching

Turborepo adalah build orchestrator. Dia urus urutan build, caching, dan paralelisasi.

Instalasi

```
npm install -D turbo
```

Konfigurasi — turbo.json

```
{
  "$schema": "https://turbo.build/schema.json",
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "outputs": ["dist/**", ".next/**"]
    },
    "dev": {
      "cache": false,
      "persistent": true
    },
    "lint": {},
    "test": {}
  }
}
```

Penjelasan:

Key	Arti
"build"	Nama task. Sama dengan script di package.json tiap workspace.
"dependsOn": ["^build"]	Build workspace dependency sebelum build workspace ini. ^ arti
"outputs"	File yang di-cache. Kalau isi source tidak berubah, Turborepo paka
"cache": false	dev tidak perlu di-cache karena persistent process.
"persistent": true	Task ini jalan terus (dev server). Jangan tunggu selesai.

Cara kerja caching

1. Turborepo hash semua input (source code + env + config).
2. Hash sama dengan sebelumnya → skip build, restore output dari cache folder.
3. Hash berubah → build ulang workspace yang berubah saja.
4. Workspace yang tidak berubah tetap pakai cache.

Ini yang bikin CI monorepo secepat multi-repo, bahkan lebih cepat karena cache lokal.

Menjalankan Turborepo

```
npm run dev          # jalanin semua dev server
turbo build          # build semua
turbo build --filter=apps/web    # build web saja
turbo build --filter=@myapp/api  # build api saja
```

5. Shared Packages — Contoh Nyata

packages/shared

Tempat tipe data, utility functions, dan konstanta yang dipakai FE & BE.

```
// packages/shared/src/types/user.ts
export interface User {
  id: string;
  name: string;
  email: string;
  role: 'admin' | 'user';
  createdAt: Date;
}

export interface CreateUserDto {
```

```

    name: string;
    email: string;
    role?: 'admin' | 'user';
  }

  export interface ApiResponse<T> {
    data: T;
    message: string;
    status: number;
  }

  // packages/shared/src/utls/validation.ts
  export function isValidEmail(email: string): boolean {
    return /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  }

  export function capitalize(str: string): string {
    return str.charAt(0).toUpperCase() + str.slice(1);
  }

```

packages/eslint-config

Config ESLint bersama. Semua project pakai aturan yang konsisten.

```

// packages/eslint-config/index.js
module.exports = {
  extends: ['next', 'next/core-web-vitals'],
  rules: {
    'no-console': 'warn',
    '@typescript-eslint/no-unused-vars': ['error', { argsIgnorePattern: '^_' }],
  },
};

// apps/web/package.json - cara pakai
{
  "eslintConfig": {
    "extends": ["@myapp/eslint-config"]
  },
  "devDependencies": {
    "@myapp/eslint-config": "*",
    "eslint": "^8.0.0"
  }
}

```

Import pattern

Setelah setup tsconfig.json dengan path alias:

```
// apps/web/tsconfig.json
{
  "compilerOptions": {
    "paths": {
      "@myapp/shared": ["../../packages/shared/src"],
      "@myapp/shared/*": ["../../packages/shared/src/*"]
    }
  }
}
```

Maka import jadi bersih:

```
// apps/web/src/pages/profile.tsx
import { User, ApiResponse, isValidEmail } from '@myapp/shared';
```

```
// apps/api/src/routes/user.ts
import { User, CreateUserDto } from '@myapp/shared';
```

6. Git Strategy untuk Monorepo

Branch strategy

```
main          → production-ready
├── develop    → integration branch
├── feat/user-crud → fitur baru
├── fix/login-bug → perbaikan bug
└── chore/update-deps → maintenance
```

Commit atomic

Satu commit satu perubahan — walaupun menyentuh banyak workspace. Ini keunggulan monorepo: refactor lintas project dalam satu commit.

```
feat(shared): add User interface with role field
feat(api): implement user CRUD endpoints
feat(web): build user profile page
```

Perubahan User di packages/shared, implementasi API, dan halaman web — semua dalam commit terpisah tapi tetap di repo sama. Tidak perlu sinkronisasi merge antar repo.

Conventional commits

Pakai format type(scope): message biar changelog dan CI bisa otomatis.

Gitignore

```
node_modules/  
dist/  
.next/  
.turbo/  
*.log  
.env  
.env.local
```

Folder `.turbo/` berisi cache Turborepo — jangan commit.

7. CI/CD untuk Monorepo

Strategi CI monorepo: **hanya build yang berubah**. Ini gak bisa pakai CI biasa yang trigger full build tiap commit. Butuh tool yang detect diff.

Turborepo + GitHub Actions (filter built-in)

Turborepo sudah include deteksi perubahan via hashing. Cukup jalankan `turbo build` — otomatis skip yang gak berubah.

Tapi kita masih perlu filter di level job agar CI gak jalan kalau gak perlu:

```
# .github/workflows/ci.yml  
name: CI  
  
on:  
  push:  
    branches: [main, develop]  
  pull_request:  
  
jobs:  
  lint:  
    runs-on: ubuntu-latest  
    steps:  
      - uses: actions/checkout@v4  
      - uses: actions/setup-node@v4  
        with:  
          node-version: 20  
      - run: npm ci  
      - run: npx turbo lint
```



```

build:
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with:
        node-version: 20
    - run: npm ci
    - run: npx turbo build

```

Strategi: turbo run --filter dengan diff

Untuk hanya build API jika file di apps/api/ dan packages/shared/ berubah:

```
turbo build --filter=$(git diff --name-only HEAD~1 | xargs -I{} dirname {} | sort)
```

Atau pakai turbo.json + inputs:

```

{
  "pipeline": {
    "build": {
      "dependsOn": ["^build"],
      "inputs": ["$TURBO_DEFAULT$", "src/**"],
      "outputs": ["dist/**"]
    }
  }
}

```

Turborepo otomatis hash hanya file dalam glob inputs. File luar (README, docs) gak trigger rebuild.

Contoh lengkap CI: build & test hanya package yang berubah dalam PR

```
name: PR Check
```

```
on: pull_request
```

```

jobs:
  changes:
    runs-on: ubuntu-latest
    outputs:
      shared: ${ steps.filter.outputs.shared }
      api: ${ steps.filter.outputs.api }
      web: ${ steps.filter.outputs.web }
    steps:

```

```

- uses: dorny/paths-filter@v3
  id: filter
  with:
    filters: |
      shared:
        - 'packages/shared/**'
      api:
        - 'apps/api/**'
        - 'packages/shared/**'
      web:
        - 'apps/web/**'
        - 'packages/shared/**'

build-api:
  needs: changes
  if: ${{ needs.changes.outputs.api == 'true' }}
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - run: npm ci
    - run: npx turbo build --filter=@myapp/api

```

Tips CI Monorepo

1. **Cache node_modules** — pakai actions/cache. Semua workspace share dep yang sama.
2. **Cache .turbo** — simpan cache Turborepo antar run biar build kilat.
3. **Parallel jobs** — lint, type-check, test, build bisa jalan bareng.
4. **Gunakan Turbo Filter** — `--filter=...` untuk target workspace spesifik.

8. Ringkasan Perintah Penting

Perintah	Fungsi
<code>npm install</code>	Install semua dep semua workspace (hoisted)
<code>npm run dev</code>	Jalankan dev server semua apps (via turbo)
<code>turbo build</code>	Build semua, cache otomatis
<code>turbo build --filter=apps/web</code>	Build web saja
<code>turbo lint</code>	Lint semua workspace
<code>turbo test</code>	Test semua, skip yang gak punya script test
<code>npx create-turbo@latest</code>	Scaffold monorepo baru

9. Kesimpulan

Monorepo dengan npm workspaces + Turborepo memberikan:

- **Shared types** — satu definisi, dipakai FE & BE. Zero duplikasi.
- **Satu CI** — konfigurasi sekali, jalan untuk semua.
- **Refactor cepat** — ganti nama field di shared, semua project kena. Satu commit.
- **Build cepat** — Turborepo cache & skip yang gak berubah.
- **Struktur rapi** — apps/ untuk eksekusi, packages/ untuk pustaka.

Cocok untuk project SMK RPL: frontend web + backend API + shared types dalam satu repo. Tim kecil bisa gercep tanpa khawatir breaking change di repo lain. Mulai dengan `npx create-turbo@latest` dan lihat sendiri bedanya.

Next module: Deploy Docker + Docker Compose untuk monorepo setup.